Project 3: From Data Cleaning to Model Implementation

Linking Physiological Responses to Cognitive Performance During Testing

A Wearable Exam Stress Dataset for Predicting Cognitive Performance in Real-World Settings

By: Melissa Bernal Garcia & Matthew Rivers

1. Abstract

Student stress during exams is a well recognized issue, however, its physiological manifestation and relationship to academic performance remain underexplored. This study investigates whether or not physiological signals recorded by a wearable device(HR, EDA, BVP, etc.) during exams can be used to predict student academic performance. Using a dataset from PhysioNet containing measurements of these signals from 10 students across 3 different exams, we constructed a performance label for each student based on average exam grades, split at the median score of 74.8 into high and low performers. Our visualizations reflected that no single physiological signal clearly seperated the two performance groups. Also constructed was a logistic regression model trained on all features, which reached a 55% accuracy score and an AUC of 0.59, both just slightly above random classification (random chance/guessing). The model identified heart rate and skin temperature as the strongest predictors of high performance. Overall, these results suggest that raw physiological signals alone may not be enough to efficiently predict academic performance. Future models and analysis would benefit from additional contextual factors such as sleep, preparation time, stress perception, as well as an increase in sample size by recruiting more students. This project also raises important considerations around the ethics of using sensitive physiological and academic data.

2. Introduction of the Data set and Variables

2.1 Data Source

The data was collected by researchers at the University of Texas at El Paso with an aim to study the relationship of physiological stress responses with academic performance. Data was collected from students during three exam sessions: Midterm 1, Midterm 2, and the Final Exam, with the use of a wearable device that tracked physiological signals.

- Participants wore an Empatica E4 wristband, an FDA-approved wearable device
- The device recorded physiological signals continuously during the exams
- Each particpant selected a device, and their identity was mapped to a device ID for later matching with grades
- After the exams, the instructor provided student grades, which were linked to the physiological data via device IDs

Each exam session started at 9:00 AM:

- Midterms lasted 1.5 hours
- Final exam lasted 3 hours

2.2 Variable Descriptions

The raw dataset consists of 7 CSV files per session across 10 subjects (students) and 3 exam sessions (30 sessions total). Before merging and pre-processing, the raw dataset included these key variables:

1. ACC: Accelerometer data, movement in 3 axes (Numeric Float)
2. BVP: Blood Volume Pulse, used to derive heart rate (Numeric Float)
3. EDA: Eelctrodermal Activity AKA skin conductance (Numeric Float)
4. HR: Heart Rate measured in beats per minute (Numeric Float)
5. IBI: Inter-Beat Interval, the time between heartbeats (Numeric Float)
6. TEMP: Skin temperature measured in degrees celcius (Numeric Float)
7. tags - Event markers/timestamps (timestamp)

During pre-processing, the working dataset has been expanded to include 348,149 rows and 16 columns, with new variables:

- hr_diff, eda_diff, acc_diff: Rate of change features derived from the HR, EDA, and ACC variables to capture how these signals shift over time (Numerical Float)
- Session: Exam session labels for each exam (Categorical Object)
- Subject: Student identifier for each student 1-10 (Categorical Object)
- high_performer: Variable for students who perform 'better' on exams, determined by those who outscore the average score for exams (Numeric Int)

Since the dataset did not include a pre-labeled stress or performance variable, but did provide the grades/scores of exams, we engineered the high_performer variable(shown above). Each student recieved grades for all 3 exams, so we normalized the scores (seen in chapter 4 during pre-processing) and derived a composite average grade for each subject (student). The median composite average grade for all subjects was 74.8, so those outperforming this score (5 subjects total) were classified as high_performers. Those scoring lower were hence classified as low performers. The high_performer variable is our **target variable**, and this project aims to identify which physiological measurements are most linked to high-performing students.

3. Research Questions

3.1: How do high-performing and low-performing students exhibit different physiological patterns during exams?

This research question aims to identify variance between EDA, BVP, and Acc. Magnitude patterns amongst low-performing students vs high-performing students. A high performing student is a student who outperforms the median test score (74.8) whereas a low performing student is a student who underperforms the median test score.

3.2: Do heart rate and skin temperature differ between high and low performers across exam sessions?

This research question aims to determine whether or not both heart rate and/or skin temperature differ between high-performing students and low-performing students. This research question will aim to be tackled within data visualizations, along with research question 1 and 3.

3.3: How do physiological signals change across the three exam sessions (Midterm 1, Midterm 2, Final)?

This research question aims to identify how different recorded physiological signals within the dataset vary across the three different exam sessions. The three exam sessions include both midterms and the final exam.

3.4: Can physiological signals measured during exams predict whether a student is a high or low performer?

This research question aims to identify whether or not physiological signals measured during exams can be used to predict whether a student is a high or low performer. We will use a logistic regression model to derive the answer and attempt to derive specific determinants of academic performance from data captured within the dataset.

4. Data Cleaning and Preprocessing

4.1 Importing Libraries, Content, and Drives

```
# import necessary libraries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import os
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount
```

```
import zipfile, os

# extract the outer zip
with zipfile.ZipFile('/content/drive/MyDrive/a-wearable-exam-stress-dataset-for-predicting-cognitive-performance-in-real
    zip_ref.extractall('/content/')

# check what came out
os.listdir('/content/')
```

```
['.config',
 'Data',
 'a-wearable-exam-stress-dataset-for-predicting-cognitive-performance-in-real-world-settings-1.0.0',
 'drive',
 'sample_data']
```

```
os.listdir('/content/a-wearable-exam-stress-dataset-for-predicting-cognitive-performance-in-real-world-settings-1.0.0')
```

```
['StudentGrades.txt', 'Data.zip', 'LICENSE.txt', 'readme.md', 'SHA256SUMS.txt']
```

```
# extract the inner Data.zip
with zipfile.ZipFile('/content/a-wearable-exam-stress-dataset-for-predicting-cognitive-performance-in-real-world-setting
    zip_ref.extractall('/content/')

# check subjects and their folders
for subject in sorted(os.listdir('/content/Data')):
    subj_path = os.path.join('/content/Data', subject)
    if os.path.isdir(subj_path):
        print(subject, ':', os.listdir(subj_path))
```

```
S1 : ['Final', 'Midterm 2', 'Midterm 1']
S10 : ['Final', 'Midterm 2', 'Midterm 1']
S2 : ['Final', 'Midterm 2', 'Midterm 1']
S3 : ['Final', 'Midterm 2', 'Midterm 1']
S4 : ['Final', 'Midterm 2', 'Midterm 1']
S5 : ['Final', 'Midterm 2', 'Midterm 1']
S6 : ['Final', 'Midterm 2', 'Midterm 1']
S7 : ['Final', 'Midterm 2', 'Midterm 1']
S8 : ['Final', 'Midterm 2', 'Midterm 1']
S9 : ['Final', 'Midterm 2', 'Midterm 1']
```

```
base_path = '/content/a-wearable-exam-stress-dataset-for-predicting-cognitive-performance-in-real-world-settings-1.0.0'
with open(f'{base_path}/StudentGrades.txt', 'r', #searched up this encoding latin thing for fix
          encoding='latin-1') as f:
    print(f.read())
```

```
GRADES - MIDTERM 1
------------------

S01    78
S02    82
S03    77
S04    75
S05    67
S06    71
S07    64
S08    92
S09    80
S10    89



GRADES - MIDTERM 2
------------------

S01    82
S02    85
S03    90
S04    77
S05    77
S06    64
S07    33
S08    88
S09    39
S10    64



GRADES - FINAL (OUT OF 200)
```

```
--------------------------

S01    182
S02    180
S03    188
S04    149
S05    157
S06    175
S07    110
S08    184
S09    126
S10    116
```

4.2 Data Pre-Proccesing and Features

```
data_path = 'Data'
```

```
#get all subject folders
subjects = os.listdir(data_path)
print(subjects)
```

```
['S4', 'S6', 'S8', 'S5', 'S1', 'S10', 'S7', 'S9', 'S3', 'S2']
```

```
# updated data_path to full dataset
data_path = '/content/Data'
base_path = '/content/a-wearable-exam-stress-dataset-for-predicting-cognitive-performance-in-real-world-settings-1.0.0'

sessions = ['Midterm 1', 'Midterm 2', 'Final']
all_subjects_data = {}

for subject in sorted(os.listdir(data_path)):
    subject_path = os.path.join(data_path, subject)
    if not os.path.isdir(subject_path):
        continue

    session_dfs = []

    for session in sessions:
        session_path = os.path.join(subject_path, session)
        if not os.path.isdir(session_path):
            continue

        try:
            # ACC
            acc = pd.read_csv(os.path.join(session_path, 'ACC.csv'))
            acc.columns = ['x', 'y', 'z']
            acc_freq = acc.iloc[0].values
            acc = acc.iloc[1:].copy().reset_index(drop=True)
            acc['Time'] = np.round(acc.index / acc_freq[0], 2)

            # EDA
            eda = pd.read_csv(os.path.join(session_path, 'EDA.csv'))
            eda.columns = ['eda']
            eda_freq = eda.iloc[0].values
            eda = eda.iloc[1:].copy().reset_index(drop=True)
            eda['Time'] = np.round(eda.index / eda_freq[0], 2)

            # BVP
            bvp = pd.read_csv(os.path.join(session_path, 'BVP.csv'))
            bvp.columns = ['bvp']
            bvp_freq = bvp.iloc[0].values
            bvp = bvp.iloc[1:].copy().reset_index(drop=True)
            bvp['Time'] = np.round(bvp.index / bvp_freq[0], 2)

            # HR
            hr = pd.read_csv(os.path.join(session_path, 'HR.csv'))
            hr.columns = ['hr']
            hr = hr.iloc[1:].copy().reset_index(drop=True)
            hr['Time'] = hr.index.astype(float)

            # TEMP
            temp = pd.read_csv(os.path.join(session_path, 'TEMP.csv'))
            temp.columns = ['temp']
            temp_freq = temp.iloc[0].values
```

```
            temp = temp.iloc[1:].copy().reset_index(drop=True)
            temp['Time'] = np.round(temp.index / temp_freq[0], 2)

            # merge
            df = hr.merge(temp, on='Time')
            df = df.merge(eda, on='Time')
            df = df.merge(bvp, on='Time')
            df = df.merge(acc, on='Time')

            df['acc_magnitude'] = np.sqrt(df['x']**2 + df['y']**2 + df['z']**2)
            df['Session'] = session
            df['Subject'] = subject

            session_dfs.append(df)

        except Exception as e:
            print(f'Error processing {subject} - {session}: {e}')

    if session_dfs:
        all_subjects_data[subject] = pd.concat(session_dfs, ignore_index=True)
```

```
# build normalized grades per subject
grades = {
    'S1':  {'m1': 78, 'm2': 82, 'final': 182},
    'S2':  {'m1': 82, 'm2': 85, 'final': 180},
    'S3':  {'m1': 77, 'm2': 90, 'final': 188},
    'S4':  {'m1': 75, 'm2': 77, 'final': 149},
    'S5':  {'m1': 67, 'm2': 77, 'final': 157},
    'S6':  {'m1': 71, 'm2': 64, 'final': 175},
    'S7':  {'m1': 64, 'm2': 33, 'final': 110},
    'S8':  {'m1': 92, 'm2': 88, 'final': 184},
    'S9':  {'m1': 80, 'm2': 39, 'final': 126},
    'S10': {'m1': 89, 'm2': 64, 'final': 116},
}

# normalize final to 100 and average all three
grade_df = pd.DataFrame(grades).T.reset_index()
grade_df.columns = ['Subject', 'm1', 'm2', 'final']
grade_df['final_norm'] = grade_df['final'] / 2
grade_df['avg_grade'] = grade_df[['m1', 'm2', 'final_norm']].mean(axis=1)

# creating a categorical variable for our target variable. target variable is high_performer, someone who outperforms th
# binary label: above median = high performer (1), below = low performer (0)
median_grade = grade_df['avg_grade'].median()
grade_df['high_performer'] = (grade_df['avg_grade'] >= median_grade).astype(int)

print(grade_df[['Subject', 'avg_grade', 'high_performer']])
print(f'\nMedian grade: {median_grade:.1f}')
```

```
  Subject  avg_grade  high_performer
0      S1  83.666667               1
1      S2  85.666667               1
2      S3  87.000000               1
3      S4  75.500000               1
4      S5  74.166667               0
5      S6  74.166667               0
6      S7  50.666667               0
7      S8  90.666667               1
8      S9  60.666667               0
9     S10  70.333333               0

Median grade: 74.8
```

```
# concat all subjects/students
final_df = pd.concat(all_subjects_data.values(), ignore_index=True)

# merge grade labels in
final_df = final_df.merge(grade_df[['Subject', 'avg_grade', 'high_performer']], on='Subject')

# remove bad EDA measurements
final_df = final_df[final_df['eda'] > 0]

# smooth noisy signals
# searched up how to do this
final_df['eda'] = final_df.groupby('Subject')['eda'].transform(lambda x: x.rolling(5).mean())
final_df['bvp'] = final_df.groupby('Subject')['bvp'].transform(lambda x: x.rolling(5).mean())
final_df['hr']  = final_df.groupby('Subject')['hr'].transform(lambda x: x.rolling(5).mean())
```

```
# remove unrealistic values / potential conclusionary inhibitive outliers
final_df = final_df[(final_df['hr'] > 40) & (final_df['hr'] < 200)]
final_df = final_df[(final_df['temp'] > 20) & (final_df['temp'] < 45)]

# diff features for coeff visualization later on + potentially model
# mainly created for the model later on
final_df['acc_diff'] = final_df.groupby('Subject')['acc_magnitude'].diff()
final_df['hr_diff']  = final_df.groupby('Subject')['hr'].diff()
final_df['eda_diff'] = final_df.groupby('Subject')['eda'].diff()

# drop NaNs
final_df = final_df.dropna()

print(final_df.shape)
print(final_df['high_performer'].value_counts())
print(final_df['Session'].value_counts())
```

```
(348149, 16)
high_performer
1    187445
0    160704
Name: count, dtype: int64
Session
Final        161496
Midterm 1     93532
Midterm 2     93121
Name: count, dtype: int64
```

```
# display new shape and info of dataset for working dataste
print('Working Dataset Shape:', final_df.shape)
```

```
Working Dataset Shape: (348149, 16)
```

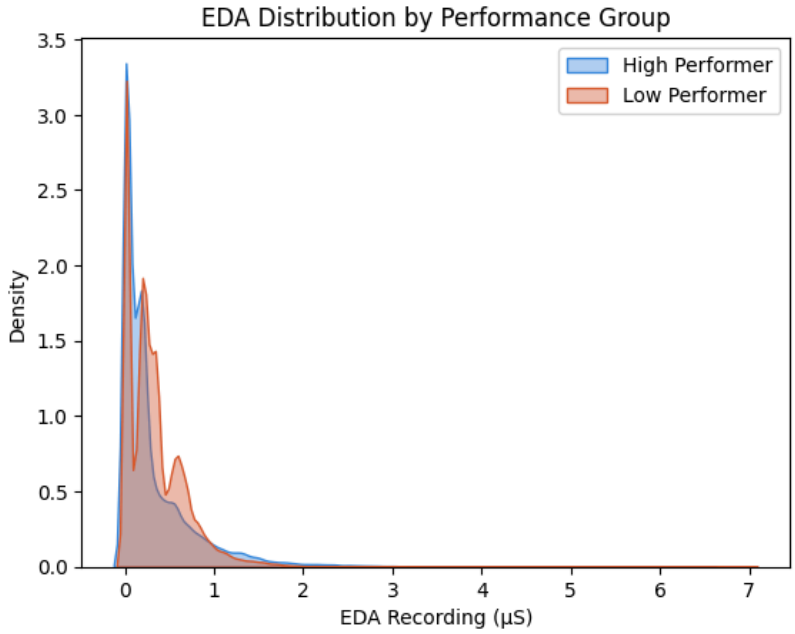
5. Visualization

5.1 - EDA Patterns Amongst High-Performing Students Compared to Low-Performing Students

5.1.1 - EDA Distribution by Performance Group

This visualization will aim to reflect the distribution in EDA measurements across different levels of test grade/performance.

```
# density plot comparing eda measurrments acros diff test performances
fig, ax = plt.subplots()
for label, color, name in [(1, '#378ADD', 'High Performer'), (0, '#D85A30', 'Low Performer')]:
    subset = final_df[final_df['high_performer'] == label]
    sns.kdeplot(subset['eda'],
                ax=ax,
                label=name,
                fill=True,
                alpha=0.4,
                color=color)
ax.set_title('EDA Distribution by Performance Group')
ax.set_xlabel('EDA Recording (µS)')
ax.set_ylabel('Density')
ax.legend()
plt.show()
```



Key Takeaways: While both groups show a strong right skew, with most EDA values nearing 0, the higher performers (students who outperformed the median of the test) show a slightly higher density peak at lower EDA values. Lower performers have a longer tail streching out closer to 1-1.5, so while there is a slight distribution pointing towards lower EDA recordings potentially yielding stronger test performance, the overlap is too close to perfectly seperate the two and defend this claim.

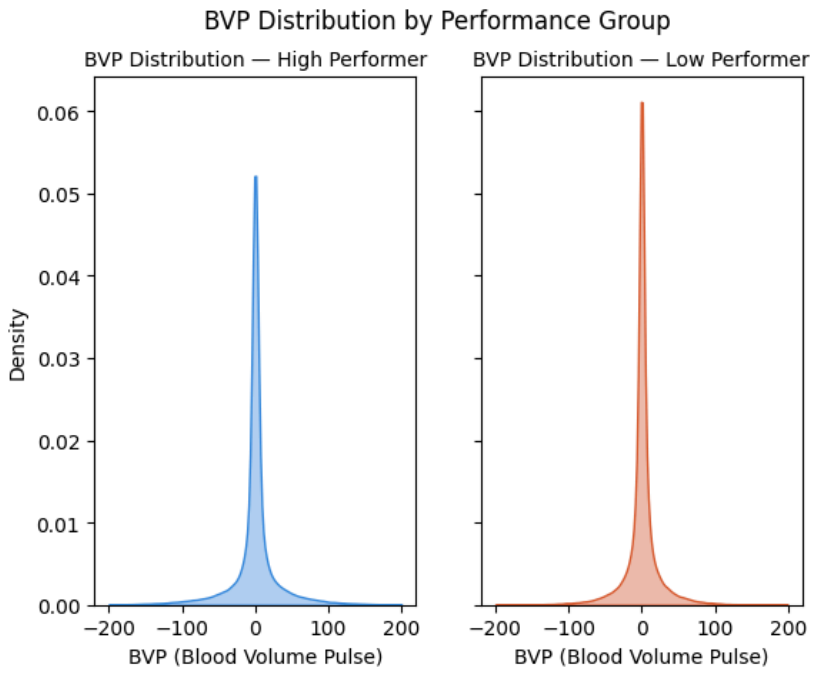
5.1.2 - BVP Distribution by Performance Group

This visualization will aim to reflect the distribution in BVP measurements across the two performance groups.

```
# bvp distribution by performance group as subplots, originally had as one plot but data is so close wnated to see 2 to
fig, axes = plt.subplots(1, 2, sharey=True)

for ax, (label, color, name) in zip(axes, [(1, '#378ADD', 'High Performer'),
                                           (0, '#D85A30', 'Low Performer')]):
    subset = final_df[final_df['high_performer'] == label]
    sns.kdeplot(subset['bvp'],
                ax=ax,
                fill=True,
                alpha=0.4,
                color=color,
                clip=(-200,200))
    # f formatting to include the high vs low peformer in title
    # fontsize 10 so we can make it smaller to fit each plot
    ax.set_title(f'BVP Distribution — {name}', fontsize = 10)
    ax.set_xlabel('BVP (Blood Volume Pulse)')
    ax.set_ylabel('Density')

plt.suptitle('BVP Distribution by Performance Group')
plt.show()
```

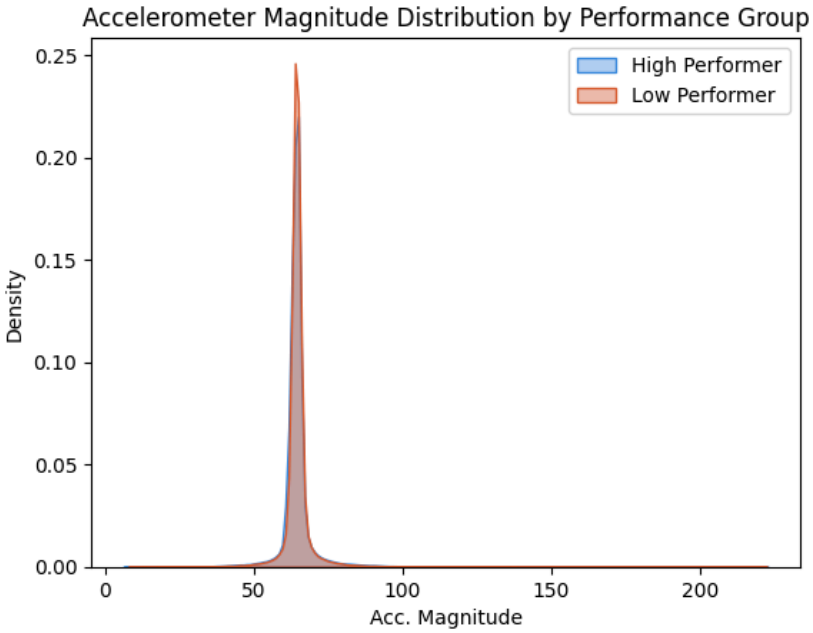


Key Takeaways: Both performance groups show a near-identical, unimodal and symmetric distribution in terms of BVP measurements. While the low performers do have a slightly higher density, the data shows the two are practically the same density measurments in terms of BVP, making BVP alone a weak predictor in terms of whether or not a student performs high or low on the exams in the dataset.

5.1.3 - Accelerometer Magnitude by Performance Group

This visualization will aim to reflect the variance of acc. magnitude measurements between the low-performance group and the high-performance group.

```
# acc. magnitude by performance group
fig, ax = plt.subplots()
for label, color, name in [(1, '#378ADD', 'High Performer'), (0, '#D85A30', 'Low Performer')]:
    subset = final_df[final_df['high_performer'] == label]
    sns.kdeplot(subset['acc_magnitude'],
                ax=ax,
                label=name,
                fill=True,
                alpha=0.4,
                color=color)
ax.set_title('Accelerometer Magnitude Distribution by Performance Group')
ax.set_xlabel('Acc. Magnitude')
ax.set_ylabel('Density')
ax.legend()
plt.show()
```



Key Takeaways: Much like the previous comparisons of BVP density between high and low performers, we can see that in terms of accelerometer magnitude distributions, the variation between high and low performers is almost non-existent. Both spreads are near identical, with the lower performers having a slightly higher spread. This may be attributed to nervousness, which can lead to fidgeting/movement during the test for the lower scoring students. However, for the most part the data is similar enough to conclude that like BVP, acc. magnitude may not be the best determinant by itself of a students performance within this dataset.

```
    fig, axes = plt.subplots(1, 3, figsize=(15, 4))

    variables = ['hr', 'eda', 'acc_magnitude']
    titles = ['Heart Rate', 'EDA', 'ACC Magnitude']

    for i, var in enumerate(variables):

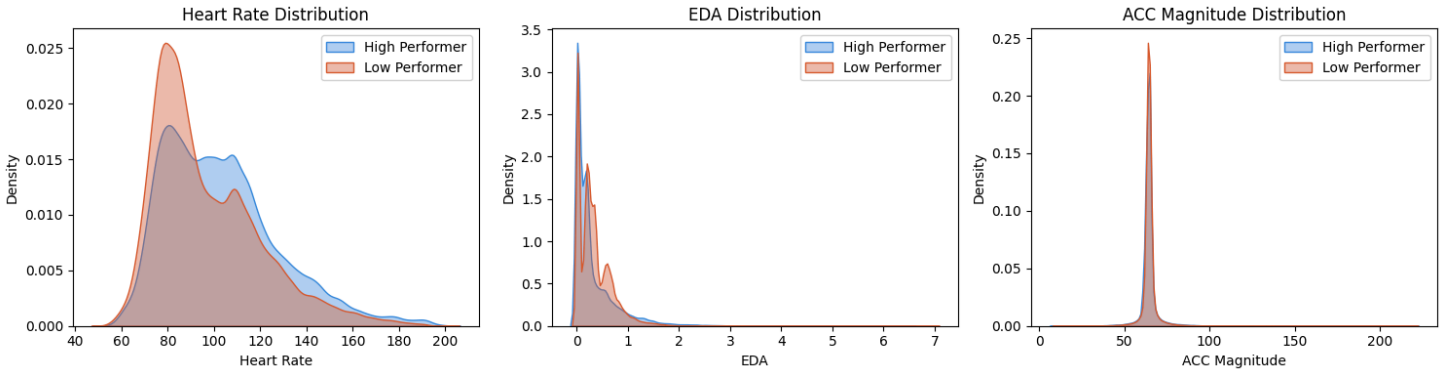
        for label, color, name in [
            (1, '#378ADD', 'High Performer'),
            (0, '#D85A30', 'Low Performer')
        ]:

            subset = final_df[final_df['high_performer'] == label]

            sns.kdeplot(
                subset[var],
                ax=axes[i],
                label=name,
                fill=True,
                alpha=0.4,
                color=color
            )

        axes[i].set_title(f'{titles[i]} Distribution')
        axes[i].set_xlabel(titles[i])
        axes[i].set_ylabel('Density')
        axes[i].legend()

    plt.tight_layout()
    plt.show()
```



Overall, high performers tend to show slightly higher and more dispersed heart rate distributions, suggesting greater variability in cardiovascular response. In EDA, low performers show more spread and higher values in mid-to-high ranges, which may indicate elevated or less regulated arousal compared to high performers. ACC magnitude distributions are more similar between groups, with only subtle shifts, implying limited separation in movement-related activity. Taken together, the strongest differences appear in heart rate and EDA, while ACC contributes less clearly to distinguishing performance groups.

5.2 - Heart Rate and Skin Temperature Variance between Performance Groups

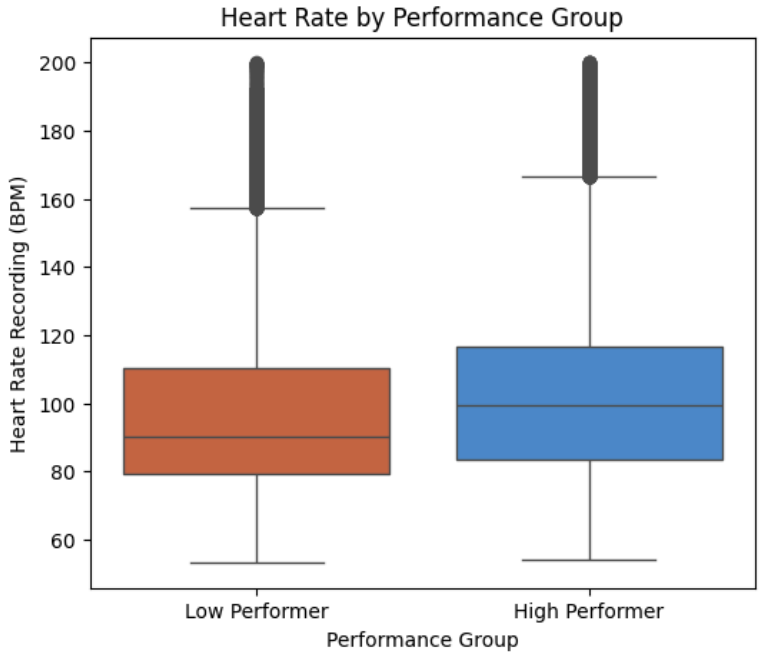
5.2.1 - HR Distribution by Performance Group

```
# heaart rate boxplot by performance group
fig, ax = plt.subplots(figsize=(6, 5))
sns.boxplot(data=final_df,
            x='high_performer',
            y='hr',
            # looked up the fix for this, had to make '0' and '1' strings rather than int
            palette={'1': '#378ADD', '0': '#D85A30'}, ax=ax)
ax.set_xticks([0, 1])
ax.set_xticklabels(['Low Performer', 'High Performer'])
ax.set_title('Heart Rate by Performance Group')
ax.set_xlabel('Performance Group')
ax.set_ylabel('Heart Rate Recording (BPM)')
plt.show()
```

```
/tmp/ipykernel_3508/372049829.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue`

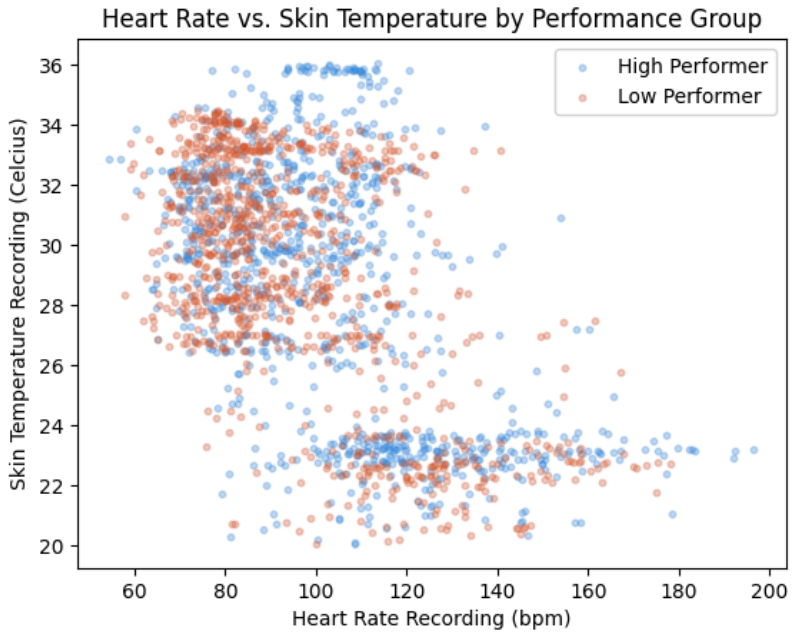
  sns.boxplot(data=final_df,
```



Key Takeaways: Heart rate distributions between low and high performers are stacked pretty close together, both groups being centered right around the 90-110 bpm range. The high performers have a bit more overall spread, as well as a slightly higher avg. bpm than the lower performers, but the similarity between the two distributions reflect that heart rate/BPM alone may not be a very efficient predictor of performance.

5.2.2 - HR vs. Skin Temp. by Performance Group

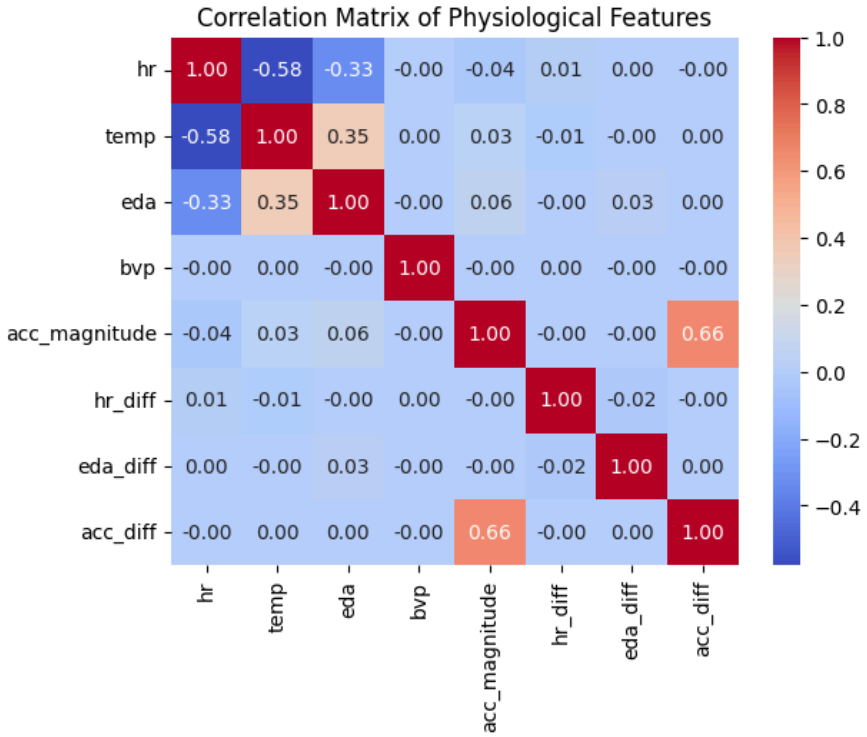
```
# scatter plot of HR vs skin temp by performance group
fig, ax = plt.subplots()
# back to int 0,1 rather than '0' '1'
for label, color, name in [(1, '#378ADD', 'High Performer'), (0, '#D85A30', 'Low Performer')]:
    subset = final_df[final_df['high_performer'] == label].sample(1000, random_state=42)
    ax.scatter(subset['hr'],
               subset['temp'],
               color=color,
               alpha=0.3,
               label=name,
               s=10)
ax.set_title('Heart Rate vs. Skin Temperature by Performance Group')
ax.set_xlabel('Heart Rate Recording (bpm)')
ax.set_ylabel('Skin Temperature Recording (Celcius)')
ax.legend()
plt.show()
```



Key Takeaways: The scatterplot above reveals two clusters, one around the low to mid 20 degrees (celcius), and another around the low 30 degrees (celcius). Within each cluster however, the data is extremely varied in terms of BPM, making it hard to determine any sort of STRONG linear relationship between the two. So, we can assume for the most part that neither feature strongly impacts the other or seperates high performers from low performers.

5.2.3 - Correlation of All Physiological Features

```
# correlation of all measured features
fig, ax = plt.subplots()
# get all features together
features = ['hr', 'temp', 'eda', 'bvp', 'acc_magnitude', 'hr_diff', 'eda_diff', 'acc_diff']
# create corr for the heatmap
corr = final_df[features].corr()
sns.heatmap(corr,
            annot=True,
            # 2 decimal places
            fmt='.2f',
            cmap='coolwarm',
            ax=ax)
ax.set_title('Correlation Matrix of Physiological Features')
plt.show()
```

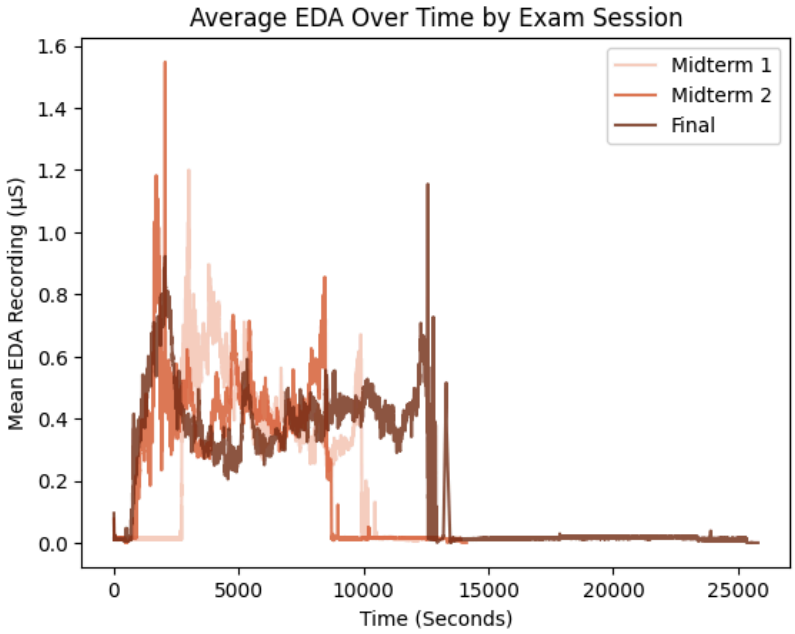


Key Takeaways: Heart rate and skin temperature reflect the strongest inverse relationship (-0.58) and EDA and skin temperature yield the strongest positive relationship (0.35). The created _diff features that reflect independent information about signal change, rather than specific levels, are negligable because of this. The strongest takeaway we can pull from this is that EDA, Skin Temp. and HR share some sort of physiological variance due to EDA's relationships with the two.

5.3 - Physiological Metric Variance Across Exams

5.3.1 - EDA by Exam

```
# avg EDA over time per exam (all 3)
fig, ax = plt.subplots()
# to differentiate with color
session_colors = {'Midterm 1': '#F5C4B3', 'Midterm 2': '#D85A30', 'Final': '#712B13'}
for session, color in session_colors.items():
    subset = final_df[final_df['Session'] == session]
    # grouping with mean rather than median since continous not discrete
    avg = subset.groupby('Time')['eda'].mean()
    ax.plot(avg.index,
            avg.values,
            color=color,
            alpha=0.8,
            label=session)
ax.set_title('Average EDA Over Time by Exam Session')
ax.set_xlabel('Time (Seconds)')
ax.set_ylabel('Mean EDA Recording (µS)')
ax.legend()
plt.show()
```

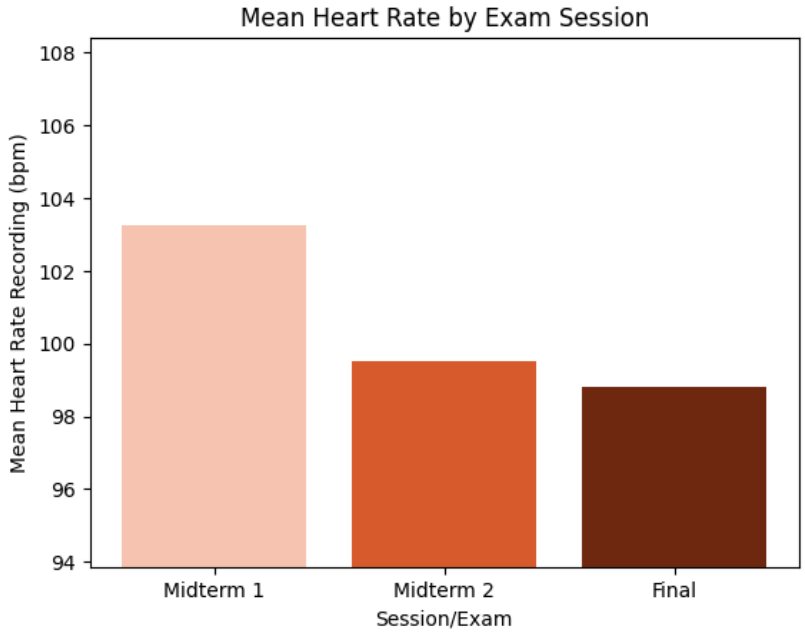


Key Takeaways: We can see that most spikes in EDA occur early into exam times, usually spiking right around 2500 seconds, followed by a steady rate around 0.5 for roughly all 3 exams. These early spikes are assumingly due to intial stress when taking an exam. However, the final exam seems to yield a longer strech of higher EDA recordings, as it's lifespan around the 0.5 mark lasts about ~2500 seconds longer than the midterms. Also notable, all 3 exams converge around near-zero EDA recordings towards the end, suggesting students potentially adapting to the exam, or more likely, finishing the exam and being still.

5.3.2 - HR by Exam

```
# avg heart rate per exam session
session_order = ['Midterm 1', 'Midterm 2', 'Final']
# again mean, since heart rate is continous not discrete
hr_means = final_df.groupby('Session')['hr'].mean().reindex(session_order)

fig, ax = plt.subplots()
ax.bar(session_order,
       hr_means.values,
       # colors for each diff exam
       color=['#F5C4B3', '#D85A30', '#712B13'])
ax.set_title('Mean Heart Rate by Exam Session')
ax.set_xlabel('Session/Exam')
ax.set_ylabel('Mean Heart Rate Recording (bpm)')
# zooms in since data is close between 98-103
ax.set_ylim(hr_means.min() * 0.95, hr_means.max() * 1.05)
plt.show()
```

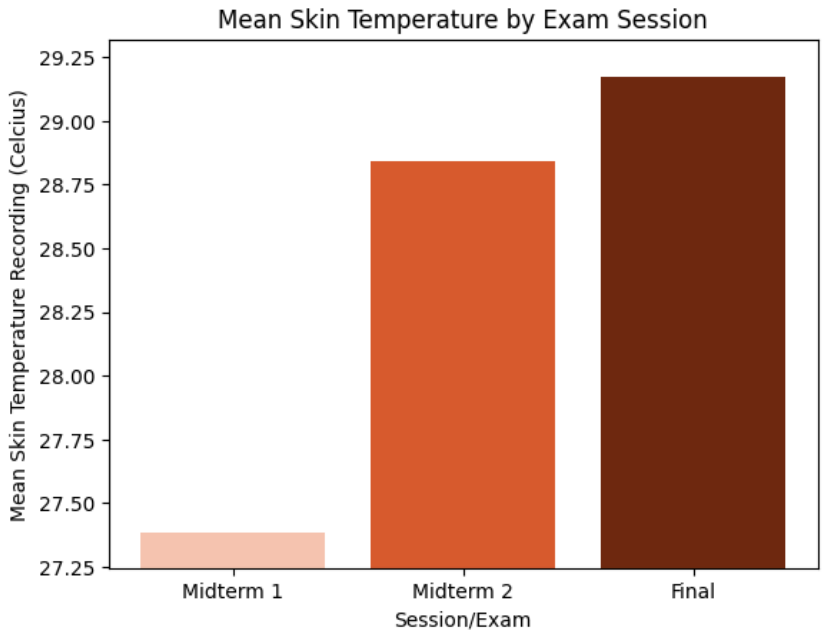


Key Takeaways: Mean heart rate is highest during midterm 1, followed by midterm 2 which is just slightly above the final exam. The downward trend over time reflects students possibly becoming more comfortable with test material, as theoretically most students can anticipate what will be included on a final whereas a midterm may be more mysterious. However, the overall variance in mean heart rates are within 10 for the 3 exams, meaning the HR differences are pretty modest rather than predictive in this context.

5.3.3 - Skin Temperature by Exam

```
# avg skin temperature per exam session
# mean again, temp is conitnous
temp_means = final_df.groupby('Session')['temp'].mean().reindex(session_order)

fig, ax = plt.subplots()
ax.bar(session_order,
       temp_means.values,
       color=['#F5C4B3', '#D85A30', '#712B13'])
ax.set_title('Mean Skin Temperature by Exam Session')
ax.set_xlabel('Session/Exam')
ax.set_ylabel('Mean Skin Temperature Recording (Celcius)')
# another zoom in
ax.set_ylim(temp_means.min() * 0.995, temp_means.max() * 1.005)
plt.show()
```



Key Takeaways: The mean skin temperature recordings across the timeline of exams rises from midterm 1 to midterm 2 to the final exam, starting at around 27.4 during the first midterm and increasing to around 29.2 for the final. This can exist for multiple reasons, even things like weather as finals typically occur more towards summer, whereas midterms usually take place in winter. Also, the declining heart rate over the exam timelines seen in the last plot may be a factor in the increasing skin temperature seen within this graph. Again however, this graph is not neccesarily informative due to the closeness of skin temperatures. Considering we still have yet to conclude a strong predictor for higher performances amongst students within all of these visualizations, we should take it a step further and begin to construct a logistic regression model to determine potential determinants.

6. Input Preparation and Normalization/Standardization

After visualizing how different physiological measurements play a role in student performance on exams, we can conclude that there isn't neccesarily a standalone measurement that we can confidently attribute to predicting performance. This supports the need for a multi-feature model, particularly in this case a **Logistic Regression** model, as our target variable (high performances) is categorical. We will begin with input preperation and data normalization/standardization.

6.1: Identifying and Treating Outliers

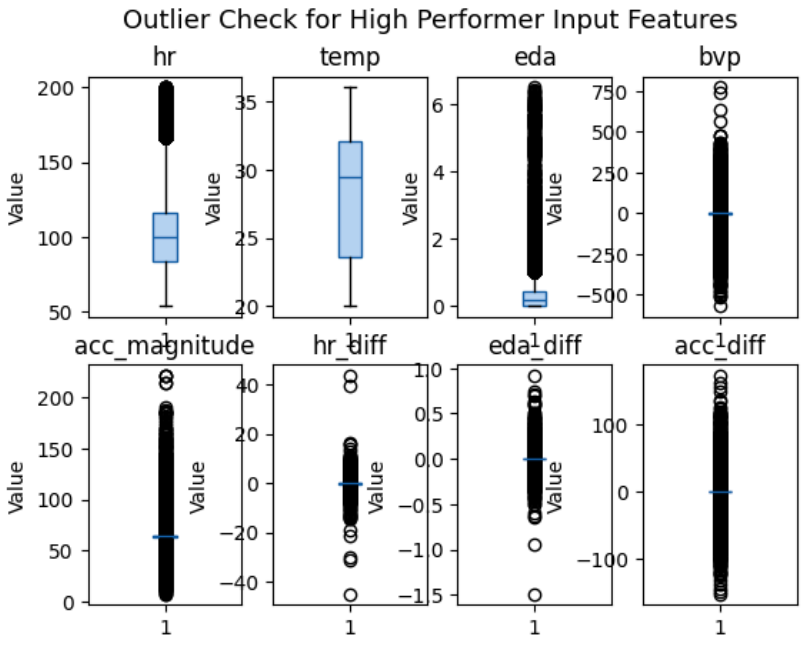
```
# visualize outliers in input features for high performers only
# creating new df for high performers, where high performs =1 as created earlier
high_df = final_df[final_df['high_performer'] == 1]

fig, axes = plt.subplots(2, 4)
axes = axes.flatten()

# listing all features recorded in dataset
features = ['hr', 'temp', 'eda', 'bvp', 'acc_magnitude', 'hr_diff', 'eda_diff', 'acc_diff']

for ax, feat in zip(axes, features):
    ax.boxplot(high_df[feat].dropna(),
               vert=True,
               patch_artist=True,
               boxprops=dict(facecolor='#B5D4F4', color='#185FA5'),
               medianprops=dict(color='#185FA5'))
    ax.set_title(feat)
    ax.set_ylabel('Value')

plt.suptitle('Outlier Check for High Performer Input Features', fontsize=13)
plt.tight_layout
plt.show()
```



Looking at this visualization, almost every single key variable has outliers, all but skin temperature. This makes sense as we saw in chap. 5.3.3 that skin temperature stayed relatively close across all 3 exam sessions. However, due to the immense amount of outliers, we will

need to further treat these outliers before modeling. We will use the IQR for each variable to cap off any outliers.

```
# making a loop to remove outliers for each varianble using IQR
# including all features/variables
featuresTreat = ['eda', 'bvp', 'acc_magnitude', 'hr_diff', 'eda_diff', 'acc_diff']
print("New Feature/Variable Ranges:")
print("")
for feature in featuresTreat:
    Q1 = final_df[feature].quantile(0.25)
    Q3 = final_df[feature].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    # capping values off
    # doing this isnetad of deleting rows (looked up how to use clip function)
    final_df[feature] = final_df[feature].clip(lower, upper)

    print(f'{feature}: lower={lower:.3f}, upper={upper:.3f} | range now: {final_df[feature].min():.5f} to {final_df[feat
```

```
New Feature/Variable Ranges:

eda: lower=-0.636, upper=1.124 | range now: 0.00128 to 1.12379
bvp: lower=-18.134, upper=18.282 | range now: -18.13400 to 18.28200
acc_magnitude: lower=59.947, upper=68.510 | range now: 59.94736 to 68.51048
hr_diff: lower=-0.870, upper=0.874 | range now: -0.87000 to 0.87400
eda_diff: lower=-0.003, upper=0.003 | range now: -0.00333 to 0.00282
acc_diff: lower=-2.732, upper=2.727 | range now: -2.73193 to 2.72680
```

Now that we have the outliers capped off, we can do the same plot to reflect the difference visually.

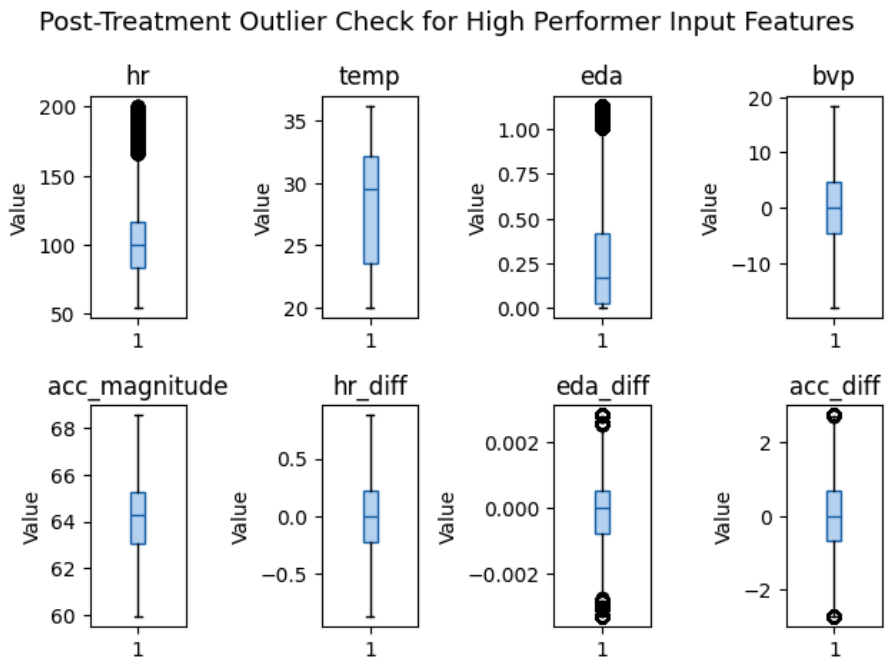
```
# re-plot boxplots after IQR outlier treatment to confirm
high_df = final_df[final_df['high_performer'] == 1]

fig, axes = plt.subplots(2, 4)
axes = axes.flatten()

features = ['hr', 'temp', 'eda', 'bvp', 'acc_magnitude', 'hr_diff', 'eda_diff', 'acc_diff']

for ax, feat in zip(axes, features):
    ax.boxplot(high_df[feat].dropna(), vert=True, patch_artist=True,
               boxprops=dict(facecolor='#B5D4F4', color='#185FA5'),
               medianprops=dict(color='#185FA5'))
    ax.set_title(feat)
    ax.set_ylabel('Value')

plt.suptitle('Post-Treatment Outlier Check for High Performer Input Features', fontsize=13)
plt.tight_layout()
plt.show()
```



Now our data is a lot cleaner with way less outliers. We can hence move towards train/test sets and then get the model.

6.2: Train/Test Set Creation

```
# importing required libraries for models/train and test
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# captuere all features
features = ['hr', 'temp', 'eda', 'bvp', 'acc_magnitude', 'hr_diff', 'eda_diff', 'acc_diff']
# X-values will be the features measured
X = final_df[features]
# y-values will be the target variable, high_performers
y = final_df['high_performer']

# defining the trains and tests
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                          test_size = 0.2,
                                         random_state = 42)

print('Training set size:', X_train.shape)
print('Test set size:', X_test.shape)
print('Value Counts per Performance Group:')
print(y_train.value_counts())
```

```
Training set size: (278519, 8)
Test set size: (69630, 8)
Value Counts per Performance Group:
high_performer
1    150028
0    128491
Name: count, dtype: int64
```

We can see that our training set is made up of 278,519 observations across all 8 key variables, and our test set is made up of 69,630 observations across all 8 key variables. Also derived is the number of value counts for each performance group, where there are 150,028 measurements for high performers (high_performer = 1) and 128,491 for low performers (high_performer = 0).

6.3: Train/Test Set Normalization

Our next step is to normalize our train/test sets. We can do this with the StandardScaler function.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Scaled X-Train Mean:")
print(X_train_scaled.mean().round(1))
print("Scaled X-Train SD:")
print(X_test_scaled.std().round(1))
```

```
Scaled X-Train Mean:
-0.0
Scaled X-Train SD:
1.0
```

Now that we have completed set normalization we can begin to construct our model.

7. Model Building

With our input features identified and outliers treated, we can now build/train our model. Our key variables, the measured physiological features within the dataset are 'X', and our target variable 'high_performer', classifying students who performed well on the exam sessions, is 'y'. We will aim to emit probabilities between 0 and 1 (ex. 0.33 = 33% probability). We then examine the model's coefficient and intercepts to understand how each feature/variable (the physiological measurements) influences the prediction (whether or not a student performs well), before generating predictions on the test set.

7.1: Initialization

```
# importing the logistic regression model
from sklearn.linear_model import LogisticRegression
# initilizing and fitting to our scaled X-train
log_reg = LogisticRegression(max_iter=1000, random_state=42)
log_reg.fit(X_train_scaled, y_train)
```

```
LogisticRegression
LogisticRegression(max_iter=1000, random_state=42)
```

Next we will find the coeffeceients for each variable/feature, as well as the intercept.

7.2: Coefficients/Intercept

```
# coefficients
print("Coefficients for Each Variable:")
print(log_reg.coef_)
print("")
# intercept
print("Model Intercept:")
print(log_reg.intercept_)
```

```
Coefficients for Each Variable:
[[ 0.31297304  0.13771672 -0.06865855 -0.00240224 -0.08989768 -0.00125602
   0.02888785  0.04454838]]

Model Intercept:
[0.1597894]
```

Next we will generate predictions on the test set.

7.3: Generating Predictions

```
# generate predictions on the test set
y_test_pred = log_reg.predict(X_test_scaled)
y_test_pred
```

```
array([1, 1, 1, ..., 1, 0, 1])
```

```
# view test labels
y_test.head()
```

	high_performer
47773	0
375860	1
16339	1
349487	0
439449	0

dtype: int64

```
# flattening arrays for ease during comparison
y_test_flat = y_test.values.ravel()
y_test_pred_flat = y_test_pred.ravel()
```

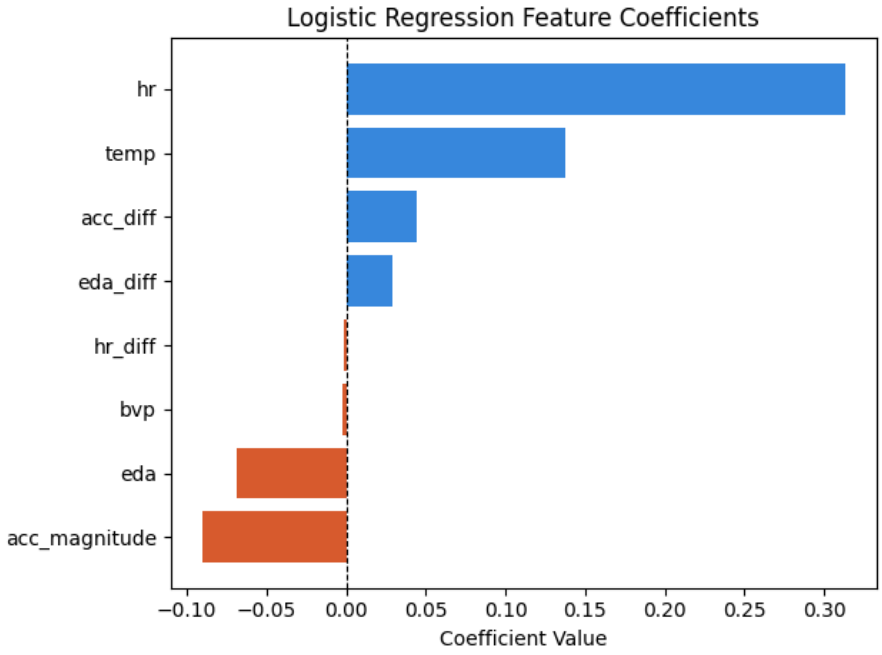
Now, we are ready to create the coefficient bar chart, that we can do performance evaluation on in chapter 8.

7.4: Coefficient Bar Chart

```
# visualize feature coefficients where:
# positive = high performer
# negative = low performer
```

```
coef_df = pd.DataFrame({
    # include all features
    'Feature': ['hr', 'temp', 'eda', 'bvp', 'acc_magnitude', 'hr_diff', 'eda_diff', 'acc_diff'],
    'Coefficient': log_reg.coef_[0]
}).sort_values('Coefficient')

fig, ax = plt.subplots()
colors = ['#D85A30' if c < 0 else '#378ADD' for c in coef_df['Coefficient']]
ax.barh(coef_df['Feature'],
        coef_df['Coefficient'],
        color=colors)
ax.axvline(0,
           color='black',
           linewidth=0.8,
           linestyle='--')
ax.set_title('Logistic Regression Feature Coefficients')
ax.set_xlabel('Coefficient Value')
plt.tight_layout()
plt.show()
```



The feature coefficient chart reveals that heart rate (HR) is by far the strongest positive predictor of high performance at a coefficient value over 0.30, followed by skin temperature (TEMP) at a coefficient value around 0.15. This suggests that high performers tend to exhibit higher heart rates and warmer skin temperatures during exams. Accelerometer magnitude and EDA show the strongest negative coefficients, meaning higher movement and skin conductance are associated with low performers. This is actually consistent with our earlier visualization in chapter 5.1.3, which showed low performers exhibiting slightly more movement reflected by accelerator magnitude. BVP and hr_diff sit near zero, confirming what the heatmap showed earlier in the reflection that these features carry minimal predictive signal on their own. Overall the coefficient pattern aligns decently with the exploratory findings from section 5, giving us confidence the model is learning meaningful physiological predictors and determinants. In chapter 8 we will perform model/performance evaluation on the test data.

8. Performance Evaluation on Test Data

8.1: Classification Metrics

8.1.1: Calculating Metrics

Since this is a logistic model, we will derive the following key metrics:

- Accuracy: percent of predictions that were correct
- Precision: how many predicted high performers were actually high performers
- Recall: how many of all actual high performers were caught
- F1 Score: balance between precision and recall

- AUC: how well the model separates the two classes

```
# importing libraries for each metric
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, classification_report)

# generate predictions/probabilities on test set
y_pred = log_reg.predict(X_test_scaled)
y_prob = log_reg.predict_proba(X_test_scaled)[:, 1]

# print all classification metrics as report
print('Metric Classifications Report:')
print(classification_report(y_test, y_pred,
      target_names=['Low Performer', 'High Performer']))

# indivisual metric values
print('Metric Values:')
print(f'Accuracy: {accuracy_score(y_test, y_pred):.4f}')
print(f'Precision: {precision_score(y_test, y_pred):.4f}')
print(f'Recall: {recall_score(y_test, y_pred):.4f}')
print(f'F1 Score: {f1_score(y_test, y_pred):.4f}')
print(f'AUC: {roc_auc_score(y_test, y_prob):.4f}')
```

```
Metric Classifications Report:
                precision    recall  f1-score   support

 Low Performer       0.52      0.38      0.44     32213
High Performer       0.57      0.70      0.63     37417

      accuracy                           0.55     69630
     macro avg       0.55      0.54      0.54     69630
  weighted avg       0.55      0.55      0.54     69630

Metric Values:
Accuracy: 0.5541
Precision: 0.5692
Recall: 0.6997
F1 Score: 0.6278
AUC: 0.5854
```

8.1.2: Metric Interpretations

- Accuracy: *0.55*
  - The model returned an accuracy value of 0.55, meaning it correctly classified 55% of the students. This is pretty low as it's almost a 50/50 shot, suggesting (as reflected and suggested by the earlier visualizations) that standalone physiological signals alone are a weak predictor of performance.
- Precision: *0.57*
  - The model returned a precision value of 0.57, meaning that when the model predicted a student was a high performer, it was correct 57% of the time. This is decent as it's correct more than half, but still not very strong being so close to a 50/50 shot similar to the accuracy value.
- Recall: *0.70*
  - The model returned a recall value of 0.70, meaning that the model caught about 70% of actual high performers. While this is pretty solid, it's at the cost of identifying some low performers incorrectly.
- F1 Score: *0.63*
  - The model returned an F1 score of 0.63, reflecting the balance between the precision and recall. This reflects that the model performs better on high performers rather than low performers.
- AUC: *0.59*
  - The model returned an AUC value of 0.59, which does show that the model is slighly better than random chance however a value above 0.80 would be much more preffered for confidence.

Note: these values within this explanation are rounded.

8.1.3: Metric Conclusions

The honest takeaway is that the model does tend to struggle, although it is slighlty better than random chance. The recall for high performers is relatively strong at 70%, however the accuracy and precision remain low, indicating the model tends to incorrectly classify a portion of low performers as high within it's correct identification of other high performers. None of the individual physiological signals CLEARLY seperated the high performers from the low performers, reflected by the model. These findings align and remain consistent with the suggestions derived from the exploratory visualizations in chapter 5, futher pushing the agenda that while physiological signals detected by wearable devices capture meaningful physiological patterns, such patters by themselves are not sufficient enough to be standalone predictors of academic performance. It's inferencable and entirely possible that other factors not captured by the wearable device, such as exam content and student history as well as subject-specific confidence when taking an exam may play into the results of academic performance.

8.2: Confusion Matrix

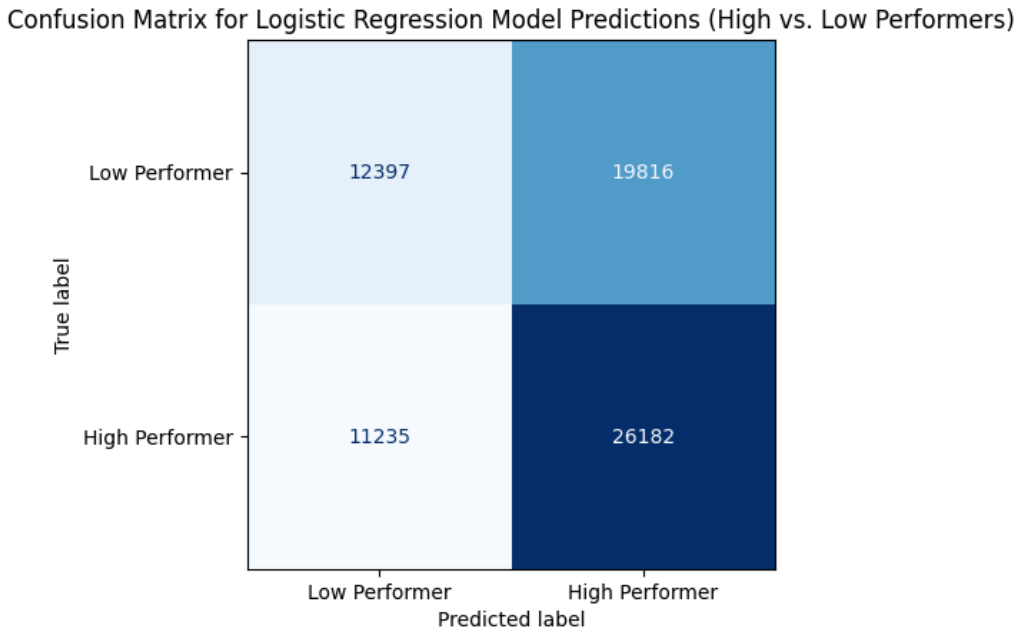
8.2.1: Constructing Confusion Matrix

We will now construct a confusion matrix to break down the model predictions into 4 catgories:

- True Negatives
  - Low performers correctly identified as low performers
- True Positives
  - High performers correctly identified as high performers
- False Negatives
  - High performers incorrectly identified as low performers
- False Positives
  - Low performers incorrectly identified as high performers

```
# importing the confusion matrix functions
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix

# constructing confusion matrix
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots()
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=['Low Performer', 'High Performer'])
disp.plot(ax=ax,
          colorbar=False,
          cmap='Blues',
          # to show actual values (one of them was 2e+04)
          values_format='d')
ax.set_title('Confusion Matrix for Logistic Regression Model Predictions (High vs. Low Performers)')
plt.show()
```



The confusion matrix above shows the breakdown of the model's test predicitons. We can see that within the model's predicitions it correctly identified 12,397 low performers (true negatives) and 26,182 high performers (true positives). However, it incorrectly identfied 11,235 high performers as low performers (false negatives) and also incorrectly identified 19,816 low performers as high performers (false positives). The high false positive account (2nd highest of all 4 portions) explains the low precision score.

8.2.2: Confusion Matrix Takeaways

While the confusion matrix highlights the fact that out of all 4 portions, the model did the best at correctly identifying and predicting true higher performers, the model faults due to it's subsequently high false positive predicitions. The conclusion we can derive from the confusion matrix, with the context of the metric scores from chapter 8.1, is that the model is better at catching actual high performers than it is at correctly rejecting low performers.

8.3: ROC Curve

8.3.1: Constructing ROC Curve

We will now construct an ROC curve to compare the model's rates between false positives and true positives. Theoretically a good model would have a line around the top left corner, as that would be a perfect AUC of 1.0, due to the fact that a random classifier (50/50) is a perfect diagnol line across the visual (AUC value of 0.50). We will include the theoretical random classifier line to compare our model's AUC to a random classifier, which would be an AUC value of 0.50.

```
# importing roc curve function
from sklearn.metrics import roc_curve

# plot ROC curve
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)

fig, ax = plt.subplots()
# our model's AUC (0.5854....)
ax.plot(fpr,
        tpr,
        color='#378ADD',
```